

CS4084

Quantum Computing

Week 13 – Lecture 25-26

Dr. Maqsood M. Khan

Department of Computer Science
FAST-NUCES Peshawar

Outline

- 1 Bridging Foundations to Algorithms
- 2 Connection: Why the Review Matters Today
- 3 Grover's Algorithm: A Visual Journey
- 4 The Evolution of Quantum Logic

Quick Review: From Circuits to Algorithms I

- **Quantum Circuits & Wires:** Graphical model where **Wires** represent Qubits and **Gates** represent Unitary operations.
- **Global Operations:** Multi-qubit gates like **CNOT** enable **entanglement**, which is the "fuel" for quantum speedup.
- **Geometry & Projection:** $\langle\phi|\psi\rangle$ measures overlap, and $P = |\psi\rangle\langle\psi|$ is used to extract probabilities—essential for reading algorithm results.
- **Compound Systems ($\otimes$):** Operations combine via the **Kronecker product**, creating the large $(\mathbb{C}^2)^{\otimes n}$ space needed for complex queries.
- **New Focus: The Query Model of Computation**
 - **The Query Gate (O_f):** A "Black Box" (Oracle) that evaluates a function $f(x)$ in a single step.
 - **Standard Picture:** We compare how many "queries" a classical computer needs versus a quantum computer to solve a specific problem.
- **Quantum Speedup Examples:**

Quick Review: From Circuits to Algorithms II

- **Deutsch's Algorithm:** Determining if a function is constant or balanced using only **one** query.
- **Phase Kickback:** A clever trick where the output of a query is "kicked" back into the phase of the input qubit—this is the secret to Deutsch-Jozsa.
- **Deutsch-Jozsa & Bernstein-Vazirani:** Solving global function properties faster than any classical "bit-by-bit" check.
- **Simon's Algorithm:** Finding a hidden pattern (s) in a function; the first proof of exponential speedup over classical difficulty.

Connection: Why the Review Matters Today

The "Why"

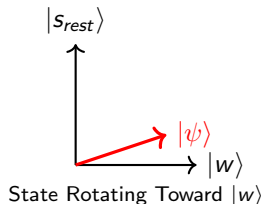
We use the **Kronecker Products** and **Unitary Matrices** from our foundations to build the **Grover Operator**. This allows us to amplify the probability of the "correct" answer while suppressing the others!

- **Last Week:** we used the Query Model to find global properties (like if a function is balanced) using the "Hadamard-Oracle-Hadamard" structure.
- **Today (Unstructured Search):** We tackle a harder problem—finding one specific "needle in a haystack" (x such that $f(x) = 1$) among N items.
- **Geometric Evolution:** Instead of just using interference once, we use our knowledge of **Geometry and Projections** to rotate the quantum state toward the correct answer using **Grover's Algorithm**.
- **Measurement:** We apply our understanding of probabilities to ensure that when we finally measure the state, we find the target item with near-certainty.

Visualizing the Search: Geometric Rotation

Grover's algorithm does not "search" by looking at items; it "rotates" the quantum state toward the right answer.

- **Initial State:** A uniform superposition of all possibilities ($H^{\otimes n}$).
- **The Oracle (Reflection 1):** Flips the sign of the "Target" state ($|w\rangle$), making it point downward.
- **The Diffusion (Reflection 2):** Flips all states about the average amplitude.



The Result

Each iteration (Grover Step) increases the probability of measuring the correct answer. We stop when the vector is perfectly aligned with the target.

Grover's Algorithm: Why It Matters

What is it? An unstructured search algorithm that finds a specific item in a database of size N .

- **Why it is Important:** Many problems in AI and CS are "Unstructured Search" problems (e.g., breaking encryption, optimizing routes, or solving Sudoku). Grover provides a tool to speed up *any* problem where you can verify the answer easily.
- **Is it available in Classical Computing? No.** Classical computers must use "Linear Search."

Feature	Classical Search	Grover's (Quantum)
Method	Check items one-by-one	Amplitude Amplification
Avg. Attempts	$N/2$	$\frac{\pi}{4}\sqrt{N}$
Complexity	$O(N)$ (Linear)	$O(\sqrt{N})$ (Quadratic Speedup)
Example ($N = 1$ Million)	500,000 queries	785 queries

The Core Difference

Classical computers are limited by the "number of looks." Quantum computers use **Quantum Interference** to process the database as a single wave, amplifying the "correct" frequency until only the answer remains.

Unstructured Search

Let $\Sigma = \{0, 1\}$ denote the binary alphabet (throughout the lesson).

Suppose we're given a function

$$f : \Sigma^n \rightarrow \Sigma$$

that we can *compute efficiently*.

Our goal is to find a *solution*, which is a binary string $x \in \Sigma^n$ for which $f(x) = 1$.

Search

Input: $f : \Sigma^n \rightarrow \Sigma$

Output: a string $x \in \Sigma^n$ satisfying $f(x) = 1$, or “no solution” if no such strings exist

This is *unstructured* search because f is arbitrary — there's *no promise* and we can't rely on it having a structure that makes finding solutions easy.

Algorithm for Solving Unstructured Search Problems

Search

Input: $f : \Sigma^n \rightarrow \Sigma$

Output: a string $x \in \Sigma^n$ satisfying $f(x) = 1$, or “no solution” if no such strings exist

Hereafter let us write

$$N = 2^n$$

By iterating through all $x \in \Sigma^n$ and evaluating f on each one, we can solve **Search** with N queries.

This is the best we can do with a **deterministic** algorithm.

Probabilistic algorithms offer minor improvements, but still require a number of queries linear in N .

Grover's algorithm is a **quantum algorithm** for **Search** requiring $O(\sqrt{N})$ queries.

Before Grover's lets Discuss Phase Query Gates

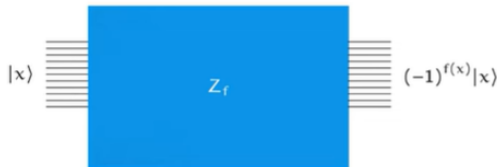
We assume that we have access to the function $f : \Sigma^n \rightarrow \Sigma$ through a query gate:

$$U_f : |\alpha\rangle|x\rangle \mapsto |\alpha \oplus f(x)\rangle|x\rangle \quad (\text{for all } \alpha \in \Sigma \text{ and } x \in \Sigma^n)$$

(We can build a circuit for U_f given a Boolean circuit for f .)

The **phase query gate** for f operates like this:

$$Z_f : |x\rangle \mapsto (-1)^{f(x)}|x\rangle \quad (\text{for all } x \in \Sigma^n)$$



Before Grover's lets Discuss Phase Query Gates (Continue..)

The *phase query gate* for f operates like this:

$$Z_f : |x\rangle \mapsto (-1)^{f(x)} |x\rangle \quad (\text{for all } x \in \Sigma^n)$$

We're also going to need a phase query gate for the n -bit OR function:

$$\text{OR}(x) = \begin{cases} 0 & x = 0^n \\ 1 & x \neq 0^n \end{cases} \quad (\text{for all } x \in \Sigma^n)$$

$$Z_{\text{OR}}|x\rangle = \begin{cases} |x\rangle & x = 0^n \\ -|x\rangle & x \neq 0^n \end{cases} \quad (\text{for all } x \in \Sigma^n)$$

Grover's Algorithm

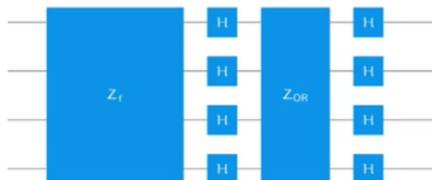
Grover's algorithm

1. *Initialize*: set n qubits to the state $H^{\otimes n}|0^n\rangle$.
2. *Iterate*: apply the **Grover operation** t times (for t to be specified later).
3. *Measure*: a standard basis measurement yields a candidate solution.

The Grover operation is defined like this:

$$G = H^{\otimes n} Z_{OR} H^{\otimes n} Z_f$$

Z_f is the phase query gate for f and Z_{OR} is the phase query gate for the n -bit OR function.



Grover's Algorithm (Continue...)

Z_f is the phase query gate for f and Z_{OR} is the phase query gate for the n -bit OR function.

A typical way that Grover's algorithm can be applied:

1. Choose the number of iterations t (next section).
2. Run Grover's algorithm with t iterations to get a candidate solution x .
3. Check the solution. If $f(x) = 1$ then output x , otherwise either run Grover's algorithm again (possibly with a different t) or report "no solutions."

Solutions and non-solutions

We'll refer to the n qubits being used for Grover's algorithm as a register Q .

We're interested in what happens when Q is initialized to the state $H^{\otimes n} |0^n\rangle$ and the Grover operation G is performed iteratively.

$$G = H^{\otimes n} Z_{\text{OR}} H^{\otimes n} Z_f$$

These are the sets of non-solutions and solutions:

$$A_0 = \{x \in \Sigma^n : f(x) = 0\}$$

$$A_1 = \{x \in \Sigma^n : f(x) = 1\}$$

We will be interested in *uniform superpositions* over these sets:

$$|A_0\rangle = \frac{1}{\sqrt{|A_0|}} \sum_{x \in A_0} |x\rangle$$

$$|A_1\rangle = \frac{1}{\sqrt{|A_1|}} \sum_{x \in A_1} |x\rangle$$

Analysis: Basic Idea

$$\begin{aligned} \mathcal{A}_0 &= \{x \in \Sigma^n : f(x) = 0\} & \mathcal{A}_1 &= \{x \in \Sigma^n : f(x) = 1\} \\ |\mathcal{A}_0\rangle &= \frac{1}{\sqrt{|\mathcal{A}_0|}} \sum_{x \in \mathcal{A}_0} |x\rangle & |\mathcal{A}_1\rangle &= \frac{1}{\sqrt{|\mathcal{A}_1|}} \sum_{x \in \mathcal{A}_1} |x\rangle \end{aligned}$$

The register Q is first initialized to this state:

$$|u\rangle = H^{\otimes n} |0^n\rangle = \frac{1}{\sqrt{N}} \sum_{x \in \Sigma^n} |x\rangle$$

This state is contained in the subspace spanned by $|\mathcal{A}_0\rangle$ and $|\mathcal{A}_1\rangle$:

$$|u\rangle = \sqrt{\frac{|\mathcal{A}_0|}{N}} |\mathcal{A}_0\rangle + \sqrt{\frac{|\mathcal{A}_1|}{N}} |\mathcal{A}_1\rangle$$

The state of Q *remains in this subspace* after every application of the Grover operation G .

Action of the Grover's Operations

We can better understand the Grover operation by splitting it into two parts:

$$G = (H^{\otimes n} Z_{\text{OR}} H^{\otimes n})(Z_f)$$

1. Recall that Z_f is defined like this:

$$Z_f|x\rangle = (-1)^{f(x)}|x\rangle \quad (\text{for all } x \in \Sigma^n)$$

Its action on $|A_0\rangle$ and $|A_1\rangle$ is simple:

$$Z_f|A_0\rangle = |A_0\rangle$$

$$Z_f|A_1\rangle = -|A_1\rangle$$

Action of the Grover Operations (Continue..)

We can better understand the Grover operation by splitting it into two parts:

$$G = (H^{\otimes n} Z_{OR} H^{\otimes n})(Z_f)$$

2. The operation Z_{OR} is defined like this:

$$Z_{OR}|x\rangle = \begin{cases} |x\rangle & x = 0^n \\ -|x\rangle & x \neq 0^n \end{cases} \quad (\text{for all } x \in \Sigma^n)$$

Here's an alternative way to express Z_{OR} :

$$Z_{OR} = 2|0^n\rangle\langle 0^n| - \mathbb{1}$$

Using this expression, we can write $H^{\otimes n} Z_{OR} H^{\otimes n}$ like this:

$$H^{\otimes n} Z_{OR} H^{\otimes n} = H^{\otimes n} (2|0^n\rangle\langle 0^n| - \mathbb{1}) H^{\otimes n} = 2|u\rangle\langle u| - \mathbb{1}$$

Action of the Grover's Operations (Continue..)

$$\begin{aligned} Z_f|A_0\rangle &= |A_0\rangle \\ Z_f|A_1\rangle &= -|A_1\rangle \end{aligned} \quad |u\rangle = \sqrt{\frac{|A_0|}{N}}|A_0\rangle + \sqrt{\frac{|A_1|}{N}}|A_1\rangle$$

$$\begin{aligned} G|A_0\rangle &= (2|u\rangle\langle u| - \mathbb{I})Z_f|A_0\rangle \\ &= (2|u\rangle\langle u| - \mathbb{I})|A_0\rangle \\ &= 2\sqrt{\frac{|A_0|}{N}}|u\rangle - |A_0\rangle \\ &= 2\sqrt{\frac{|A_0|}{N}}\left(\sqrt{\frac{|A_0|}{N}}|A_0\rangle + \sqrt{\frac{|A_1|}{N}}|A_1\rangle\right) - |A_0\rangle \\ &= \frac{|A_0| - |A_1|}{N}|A_0\rangle + \frac{2\sqrt{|A_0| \cdot |A_1|}}{N}|A_1\rangle \end{aligned}$$

Action of the Grover's Operations (Continue..)

$$\begin{aligned} Z_f|A_0\rangle &= |A_0\rangle \\ Z_f|A_1\rangle &= -|A_1\rangle \end{aligned} \quad |u\rangle = \sqrt{\frac{|A_0|}{N}}|A_0\rangle + \sqrt{\frac{|A_1|}{N}}|A_1\rangle$$

$$G|A_0\rangle = \frac{|A_0| - |A_1|}{N}|A_0\rangle + \frac{2\sqrt{|A_0| \cdot |A_1|}}{N}|A_1\rangle$$

$$\begin{aligned} G|A_1\rangle &= (2|u\rangle\langle u| - \mathbb{I})Z_f|A_1\rangle \\ &= (\mathbb{I} - 2|u\rangle\langle u|)|A_1\rangle \\ &= |A_1\rangle - 2\sqrt{\frac{|A_1|}{N}}|u\rangle \\ &= |A_1\rangle - 2\sqrt{\frac{|A_0|}{N}}\left(\sqrt{\frac{|A_0|}{N}}|A_0\rangle + \sqrt{\frac{|A_1|}{N}}|A_1\rangle\right) \\ &= -\frac{2\sqrt{|A_0| \cdot |A_1|}}{N}|A_0\rangle + \frac{|A_0| - |A_1|}{N}|A_1\rangle \end{aligned}$$

Action of the Grover's Operations (Continue..)

$$\begin{aligned} Z_f|A_0\rangle &= |A_0\rangle \\ Z_f|A_1\rangle &= -|A_1\rangle \end{aligned} \quad |u\rangle = \sqrt{\frac{|A_0|}{N}}|A_0\rangle + \sqrt{\frac{|A_1|}{N}}|A_1\rangle$$

$$\begin{aligned} G|A_0\rangle &= \frac{|A_0| - |A_1|}{N}|A_0\rangle + \frac{2\sqrt{|A_0| \cdot |A_1|}}{N}|A_1\rangle \\ G|A_1\rangle &= -\frac{2\sqrt{|A_0| \cdot |A_1|}}{N}|A_0\rangle + \frac{|A_0| - |A_1|}{N}|A_1\rangle \end{aligned}$$

The action of G on $\text{span}\{|A_0\rangle, |A_1\rangle\}$ can be described by a 2×2 matrix:

$$M = \begin{pmatrix} \frac{|A_0| - |A_1|}{N} & -\frac{2\sqrt{|A_0| \cdot |A_1|}}{N} \\ \frac{2\sqrt{|A_0| \cdot |A_1|}}{N} & \frac{|A_0| - |A_1|}{N} \end{pmatrix} \begin{matrix} |A_0\rangle \\ |A_1\rangle \end{matrix}$$

$|A_0\rangle \qquad \qquad |A_1\rangle$

Rotation by an Angle

The action of G on $\text{span}\{|A_0\rangle, |A_1\rangle\}$ can be described by a 2×2 matrix:

$$M = \begin{pmatrix} \frac{|A_0| - |A_1|}{N} & -\frac{2\sqrt{|A_0| \cdot |A_1|}}{N} \\ \frac{2\sqrt{|A_0| \cdot |A_1|}}{N} & \frac{|A_0| - |A_1|}{N} \end{pmatrix} = \begin{pmatrix} \sqrt{\frac{|A_0|}{N}} & -\sqrt{\frac{|A_1|}{N}} \\ \sqrt{\frac{|A_1|}{N}} & \sqrt{\frac{|A_0|}{N}} \end{pmatrix}^2$$

This is a **rotation** matrix.

$$\begin{pmatrix} \sqrt{\frac{|A_0|}{N}} & -\sqrt{\frac{|A_1|}{N}} \\ \sqrt{\frac{|A_1|}{N}} & \sqrt{\frac{|A_0|}{N}} \end{pmatrix} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix} \quad \theta = \sin^{-1}\left(\sqrt{\frac{|A_1|}{N}}\right)$$

$$M = \begin{pmatrix} \cos(2\theta) & -\sin(2\theta) \\ \sin(2\theta) & \cos(2\theta) \end{pmatrix}$$

Rotation by an Angle (Continue...)

$$M = \begin{pmatrix} \cos(2\theta) & -\sin(2\theta) \\ \sin(2\theta) & \cos(2\theta) \end{pmatrix} \quad \theta = \sin^{-1} \left(\sqrt{\frac{|A_1|}{N}} \right)$$

After the initialization step, this is the state of the register Q:

$$|u\rangle = \sqrt{\frac{|A_0|}{N}} |A_0\rangle + \sqrt{\frac{|A_1|}{N}} |A_1\rangle = \cos(\theta) |A_0\rangle + \sin(\theta) |A_1\rangle$$

Each time the Grover operation G is performed, the state of Q is rotated by an angle 2θ :

$$\begin{aligned} |u\rangle &= \cos(\theta) |A_0\rangle + \sin(\theta) |A_1\rangle \\ G|u\rangle &= \cos(3\theta) |A_0\rangle + \sin(3\theta) |A_1\rangle \\ G^2|u\rangle &= \cos(5\theta) |A_0\rangle + \sin(5\theta) |A_1\rangle \\ &\vdots \\ G^t|u\rangle &= \cos((2t+1)\theta) |A_0\rangle + \sin((2t+1)\theta) |A_1\rangle \end{aligned}$$

Geometric representations of Grover's Algorithm

Main idea

The operation $G = H^{\otimes n} Z_{\text{OR}} H^{\otimes n} Z_f$ is a composition of **two reflections**:

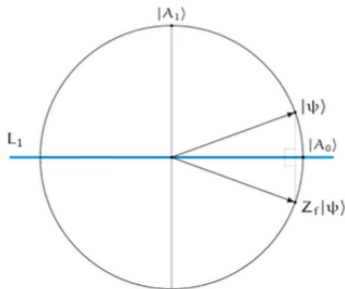
$$Z_f \text{ and } H^{\otimes n} Z_{\text{OR}} H^{\otimes n}$$

Composing two reflections yields a **rotation**.

1. Recall that Z_f has this action on the 2-dimensional space spanned by $|A_0\rangle$ and $|A_1\rangle$:

$$\begin{aligned} Z_f |A_0\rangle &= |A_0\rangle \\ Z_f |A_1\rangle &= -|A_1\rangle \end{aligned}$$

This is a **reflection** about the line L_1 parallel to $|A_0\rangle$.

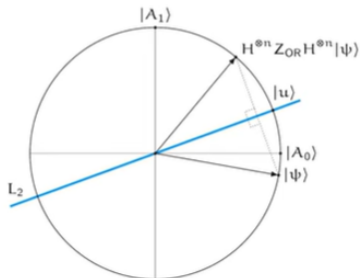


Geometric representations of Grover's Algorithm (Continue...)

2. The operation $H^{\otimes n} Z_{\text{OR}} H^{\otimes n}$ can be expressed like this:

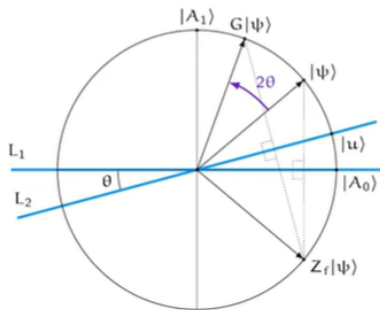
$$H^{\otimes n} Z_{\text{OR}} H^{\otimes n} = 2|u\rangle\langle u| - \mathbb{I}$$

Again this is a **reflection**, this time about the line L_2 parallel to $|u\rangle$.



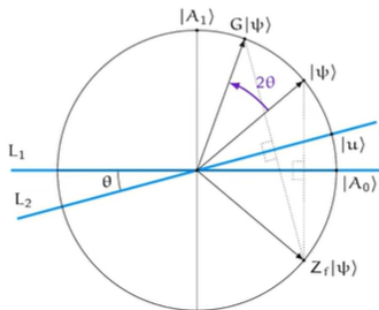
Geometric representations of Grover's Algorithm (Continue...)

When we compose two reflections, we obtain a **rotation** by twice the angle between the lines of reflection.



Number of Iterations in choosing the Grovers Algorithm

When we compose two reflections, we obtain a **rotation** by twice the angle between the lines of reflection.



Setting the target

Consider any quantum state of this form:

$$\alpha|A_0\rangle + \beta|A_1\rangle$$

Measuring yields a solution $x \in A_1$ with probability $|\beta|^2$.

$$\alpha|A_0\rangle + \beta|A_1\rangle = \frac{\alpha}{\sqrt{|A_0|}} \sum_{x \in A_0} |x\rangle + \frac{\beta}{\sqrt{|A_1|}} \sum_{x \in A_1} |x\rangle$$

$$p(x) = \begin{cases} \frac{|\alpha|^2}{|A_0|} & x \in A_0 \\ \frac{|\beta|^2}{|A_1|} & x \in A_1 \end{cases}$$

$$\Pr(\text{outcome is in } A_1) = \sum_{x \in A_1} p(x) = |\beta|^2$$

Number of Iterations in choosing the Grovers Algorithm(Continue...)

Consider any quantum state of this form:

$$\alpha|A_0\rangle + \beta|A_1\rangle$$

Measuring yields a solution $x \in A_1$ with probability $|\beta|^2$.

The state of Q after t iterations in Grover's algorithm:

$$\cos((2t+1)\theta)|A_0\rangle + \sin((2t+1)\theta)|A_1\rangle \quad \theta = \sin^{-1}\left(\sqrt{\frac{|A_1|}{N}}\right)$$

Measuring after t iterations gives an outcome $x \in A_1$ with probability

$$\sin^2((2t+1)\theta)$$

We wish to maximize this probability – so we may view that $|A_1\rangle$ is our **target state**.

Number of Iterations in choosing the Grovers Algorithm (Continue...)

The state of Q after t iterations in Grover's algorithm:

$$\cos((2t + 1)\theta)|A_0\rangle + \sin((2t + 1)\theta)|A_1\rangle \quad \theta = \sin^{-1}\left(\sqrt{\frac{|A_1|}{N}}\right)$$

Measuring after t iterations gives an outcome $x \in A_1$ with probability

$$\sin^2((2t + 1)\theta)$$

To make this probability close to 1 and minimize t , we will aim for

$$(2t + 1)\theta \approx \frac{\pi}{2} \quad \Leftrightarrow \quad t \approx \frac{\pi}{4\theta} - \frac{1}{2} \quad \xrightarrow{\text{closest integer}} \quad t = \left\lfloor \frac{\pi}{4\theta} \right\rfloor$$

Important considerations:

- t must be an integer
- θ depends on the number of solutions $s = |A_1|$

Unique Search

$$(2t + 1)\theta \approx \frac{\pi}{2} \quad \Leftarrow \quad t = \left\lfloor \frac{\pi}{4\theta} \right\rfloor$$

Unique search

Input: $f : \Sigma^n \rightarrow \Sigma$

Promise: There is exactly one string $z \in \Sigma^n$ for which $f(z) = 1$,
with $f(x) = 0$ for all strings $x \neq z$

Output: The string z

For Unique search we have $s = |A_1| = 1$ and therefore

$$\theta = \sin^{-1}\left(\sqrt{\frac{1}{N}}\right) \approx \sqrt{\frac{1}{N}}$$

Substituting $\theta \approx 1/\sqrt{N}$ into our expression for t gives

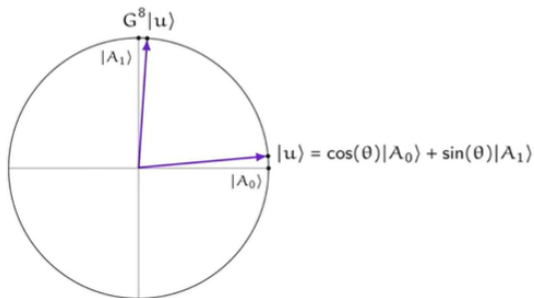
$$t \approx \left\lfloor \frac{\pi}{4} \sqrt{N} \right\rfloor \quad \Leftarrow O(\sqrt{N}) \text{ queries}$$

Unique Search (Example)

Example: $N = 128$

$$\theta = \sin^{-1}\left(\frac{1}{\sqrt{N}}\right) = 0.0885\dots$$

$$t = \left\lfloor \frac{\pi}{4\theta} \right\rfloor = 8$$



Probabilities for different numbers of N

$$\theta = \sin^{-1}\left(\sqrt{\frac{1}{N}}\right) \quad t = \left\lfloor \frac{\pi}{4\theta} \right\rfloor$$

Measuring after t iterations gives the (unique) outcome $x \in A_1$ with probability

$$p(N, 1) = \sin^2((2t + 1)\theta)$$

Success probabilities for Unique search

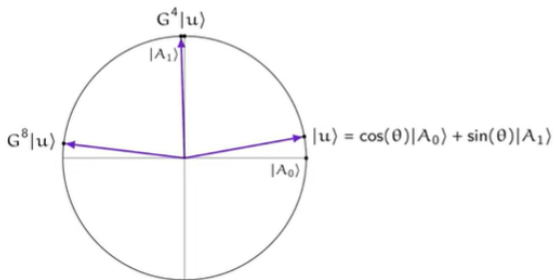
N	p(N, 1)	N	p(N, 1)
2	.5	128	.9956199
4	1.0	256	.9999470
8	.9453125	512	.9994480
16	.9613190	1024	.9994612
32	.9991823	2048	.9999968
64	.9965857	4096	.9999453

Multiple Solutions..

Example: $N = 128$, $s = 4$

$$\theta = \sin^{-1}\left(\sqrt{\frac{s}{N}}\right) = 0.1777\dots$$

$$t = \left\lfloor \frac{\pi}{4\theta} \right\rfloor = 4$$



Number of queries

$$\theta = \sin^{-1}\left(\sqrt{\frac{s}{N}}\right) \quad t = \left\lfloor \frac{\pi}{4\theta} \right\rfloor$$

Each iteration of Grover's algorithm requires 1 query (or evaluations of f). How does the number of queries t depend on N and s ?

$$\sin^{-1}(x) \geq x \quad (\text{for every } x \in [0, 1])$$

$$\theta = \sin^{-1}\left(\sqrt{\frac{s}{N}}\right) \geq \sqrt{\frac{s}{N}}$$

$$t \leq \frac{\pi}{4\theta} \leq \frac{\pi}{4} \sqrt{\frac{N}{s}}$$

$$t = O\left(\sqrt{\frac{N}{s}}\right)$$

Unknown number of solutions

What do we do if we don't know the number of solutions in advance?

A simple approach

Choose the number of iterations $t \in \{1, \dots, \lfloor \pi\sqrt{N}/4 \rfloor\}$ *uniformly at random*.

- The probability to find a solution (if one exists) will be at least 40%.
(Repeat several times to boost success probability.)
- The number of queries (or evaluations of f) is $O(\sqrt{N})$.

A more sophisticated approach

1. Set $T = 1$.
2. Run Grover's algorithm with $t \in \{1, \dots, T\}$ chosen uniformly at random.
3. If a solution is found, output it and stop.
Otherwise, increase T and return to step 2 (or report "no solution").

- The rate of increase of T must be carefully balanced: slower rates require more queries, higher rates decrease success probability. $T \leftarrow \lceil \frac{5}{4} T \rceil$ works.
- If the number of solutions is $s \geq 1$, then the number of queries (or evaluations of f) required is $O(\sqrt{N/s})$. If there are no solutions, $O(\sqrt{N})$ queries are required.

The Path to Grover: A Logical Flow

- 1 **Single Quantum Systems:** We started with the **Qubit** ($|0\rangle, |1\rangle$). We learned that a single wire can hold a superposition, but it cannot solve complex problems alone.
- 2 **Compound Systems & Tensor Products ($\otimes$):** To process more data, we combined qubits. The **Kronecker Product** allowed us to build the large $(\mathbb{C}^2)^{\otimes n}$ space, giving us the "room" to store all 2^n possibilities simultaneously.
- 3 **The Query Model (Oracles):** With our state space ready, we introduced the **Black Box** (O_f). We learned to treat functions as physical gates that we can interact with using quantum states.
- 4 **Global Properties (Deutsch-Jozsa/Simon's):** We used **Phase Kickback** and interference to answer "What type of function is this?" or "What is the secret pattern?" in just a few queries.
- 5 **Unstructured Search (Grover's Algorithm):** Now, we reach the peak of search efficiency. We aren't just looking for a pattern; we are using **Amplitude Amplification** to pluck a single "needle" out of an unsorted haystack.